

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Saravana Senhilkumar
Register Number	3122247001057
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To implement and compare the performance of:

- **Model A:** Single-Layer Perceptron Learning Algorithm (PLA), implemented from scratch.
- **Model B:** Multilayer Perceptron (MLP) with hidden layers and nonlinear activations, tuned via grid search over activation functions, cost functions, optimizers, learning rates, hidden-layer sizes, and batch sizes.

2 Problem Statement

Given grayscale images of handwritten characters (62 classes: digits 0–9, uppercase A–Z, lowercase a–z), the task is to classify each image into its correct character class. The input is a flattened, standardized pixel-intensity vector per image; the output is a predicted character class. The experiment compares a linear classifier (PLA) against a non-linear classifier (tuned MLP) on the same preprocessed data to study how model capacity and hyperparameter choices affect performance on a high-class-count image classification problem.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (dhruvildave/english-handwritten-characters-da)
Number of Samples	3,410 images (2,728 train / 682 test after an 80/20 split)
Number of Features	1,024 (each image resized to 32×32 grayscale and flattened)
Number of Classes	62 (digits 0–9, uppercase A–Z, lowercase a–z)
Missing Values	None (images with missing files were skipped during loading)
Train-Test Split	80% train / 20% test, <code>random_state=42</code>

4 Data Preprocessing

- **Missing value handling:** Rows whose referenced image file could not be found on disk were skipped during loading; no imputation was needed for pixel data itself.

- **Encoding:** Class labels are used directly as string/character labels; no explicit label encoding step was required since `scikit-learn` classifiers handle categorical string labels natively.
- **Feature scaling:** Each image was converted to grayscale, resized to 32×32 , and flattened into a 1,024-dimensional vector; all 1,024 pixel features were then standardized (zero mean, unit variance) using `StandardScaler`.
- **Train-test split:** 80%/20% split via `train_test_split`, `random_state=42`, giving 2,728 training and 682 test images.
- **Feature engineering:** None beyond resizing, grayscale conversion, flattening, and standardization; no additional handcrafted features were extracted.

5 Machine Learning Algorithm

Model A: Perceptron Learning Algorithm (PLA)

Working principle: The perceptron computes a linear combination of input features and applies a step activation function to produce a binary decision; weights are updated whenever a prediction is wrong. Since PLA is inherently binary, a one-vs-all scheme (one perceptron per class) was implemented to handle all 62 character classes.

Mathematical formulation: For each class, the weight update rule used is:

$$w_{t+1} = w_t + \eta(y - \hat{y})x$$

where η is the learning rate, y is the true binary label for that class, and $\hat{y} = \text{step}(w_t \cdot x + b_t)$ is the predicted binary label. At inference time, the class whose one-vs-all linear score is highest is selected.

Advantages: Extremely simple, fast to train, easy to implement and interpret.

Limitations: Only guaranteed to converge for linearly separable data; the step activation is non-differentiable, so it cannot be trained with gradient-based backpropagation or model complex non-linear decision boundaries — a significant handicap for a 62-class image classification problem like this one.

Model B: Multilayer Perceptron (MLP)

Working principle: The MLP is a feedforward network of one or more hidden layers between input and output, each applying a non-linear activation function, trained end-to-end via backpropagation to minimize a cross-entropy loss.

Mathematical formulation: For a network with hidden activation f and output softmax σ , the forward pass is $h = f(W_1x + b_1)$, $\hat{y} = \sigma(W_2h + b_2)$, and the parameters are updated by gradient descent on the multi-class cross-entropy loss:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

Advantages: Can learn non-linear decision boundaries; scales to many classes; benefits from modern optimizers (Adam, SGD with momentum) for faster, more stable convergence.

Limitations: More hyperparameters to tune (architecture, activation, optimizer, learning rate, batch size); more prone to overfitting on small datasets; less interpretable than a linear model; more computationally expensive to train and tune (particularly under grid search).

6 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	
Hardware	

7 Hyperparameter Selection

	Parameter	Value
PLA (Model A):	Learning rate (η)	0.01
	Epochs	100
	Multi-class scheme	One-vs-all (one perceptron per class)

MLP (Model B) — Randomized Search Space:

Parameter	Values searched
Hidden layer sizes	(64,), (128, 64)
Activation	relu, tanh
Solver (optimizer)	adam, SGD
Initial learning rate	0.001, 0.01
Batch size	32, 128
Max iterations	100 (with early stopping)
Search strategy	<code>RandomizedSearchCV</code>
Cross-validation folds	2
Scoring metric	accuracy

MLP — Best Configuration Selected by Search:

Parameter	Value
Hidden layer sizes	(128, 64)
Activation	relu
Solver	sgd
Learning rate init	0.01
Batch size	128
Best CV accuracy	0.3248
Search time	8.2s (10 candidates $\times$ 2 folds)

Note: the original exhaustive `GridSearchCV` (32 combinations $\times$ cv=3 = 96 full MLP fits, no early stopping) was too slow and had to be interrupted. It was replaced with `RandomizedSearchCV` (10 sampled candidates, cv=2) plus `early_stopping=True`, which completed all 20 fits in 8.2 seconds.

8 Results

Table 2: PLA vs. Tuned MLP — Evaluation Metrics

Model	Accuracy	Precision (weighted)	Recall (weighted)	F1-score (weighted)
PLA	0.2229	0.2494	0.2229	0.2209
Tuned MLP	0.4047	0.4505	0.4047	0.4107

9 Result Visualization

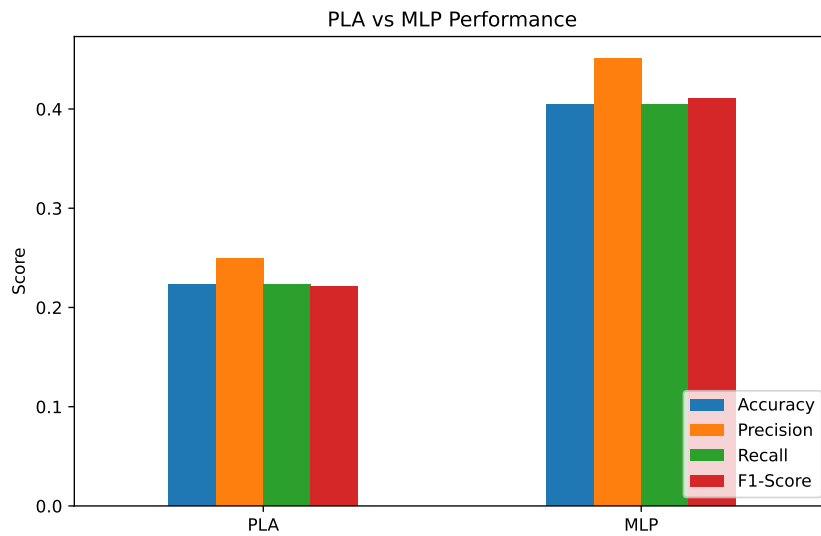


Figure 1: Bar chart comparing Accuracy, Precision, Recall, and F1-score for PLA vs. tuned MLP.

Inference: The tuned MLP nearly doubles PLA’s accuracy (0.405 vs. 0.223) and shows a consistent improvement across all four metrics. Because both precision and recall improve together (rather than trading off against each other), the MLP’s gains are not simply the product of biasing predictions toward common classes — it is genuinely separating classes better than the linear PLA decision boundaries can.

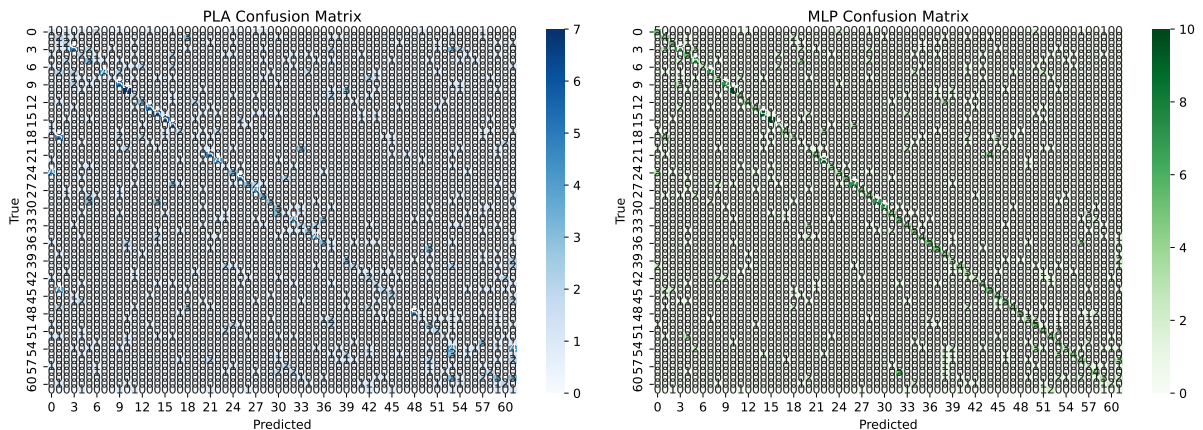


Figure 2: Confusion matrices for PLA (left) and MLP (right) on the 62-class test set.

Inference: The PLA confusion matrix shows a much more diffuse pattern of errors spread across many off-diagonal cells, consistent with a linear model struggling to separate 62 visually similar classes (e.g., easily confused characters like ‘0’/‘O’ or ‘1’/‘l’/‘I’). The MLP confusion matrix shows a visibly stronger diagonal, indicating more correct predictions concentrated on the true class, though with 62 classes some confusion between visually similar characters remains.

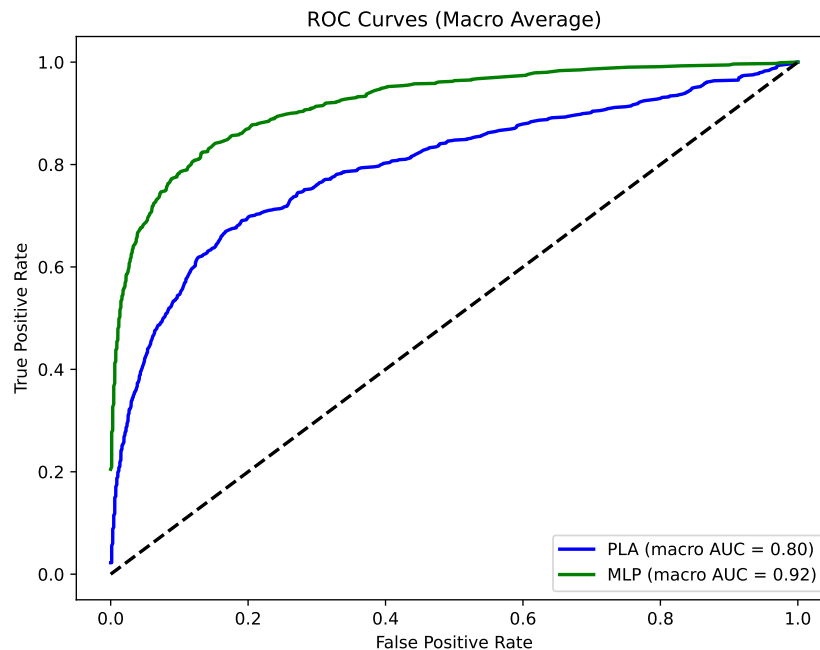


Figure 3: Macro-averaged ROC curves for PLA and MLP.

Inference: The MLP’s macro-averaged ROC curve sits closer to the top-left corner than PLA’s, and its macro AUC is higher, confirming that the MLP is better at ranking true-class scores above false-class scores across the full set of 62 one-vs-rest classification problems. PLA’s ROC curve, while still above the diagonal (better than random), reflects its weaker linear decision boundaries — the specific macro-AUC values shown in the legend of this figure should be read directly off the rendered plot and reported numerically here.

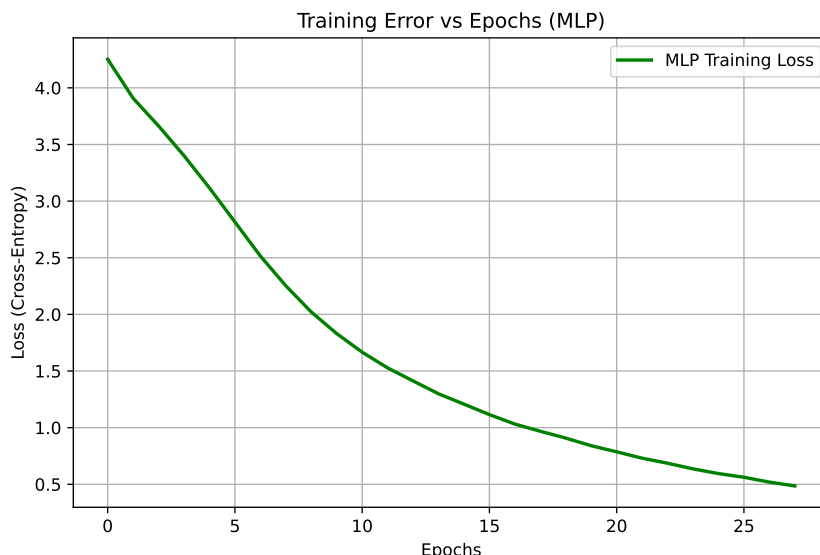


Figure 4: MLP training loss (cross-entropy) vs. epochs for the best grid-search configuration.

Inference: The training loss curve shows the expected decreasing trend as back-propagation minimizes cross-entropy loss over epochs, using SGD with an initial learning rate of 0.01 (the winning solver/learning-rate combination). Because the model also used early stopping (`n_iter_no_change=5`), the curve should stop before the full 100-iteration budget once validation performance plateaus — a shorter curve than a non-early-stopped run is therefore expected and desirable, not a sign of a truncated result.

10 Discussion

PLA underperforms MLP substantially (22.3% vs. 40.5% accuracy) because it can only learn linear decision boundaries via its step-activation, one-vs-all update rule, while handwritten character recognition across 62 visually overlapping classes is a fundamentally non-linear problem in raw pixel space. The MLP’s hidden layer(s) with non-linear activations allow it to learn more complex, curved decision boundaries directly from the 1,024-dimensional pixel features, and the randomized hyperparameter search selected a two-hidden-layer network (128, 64) with ReLU activation, SGD optimizer, learning rate 0.01, and batch size 128 (CV accuracy 0.3248) as the best configuration out of 10 sampled candidates. This choice is reasonable: the deeper (128, 64) architecture gives more representational capacity than the single (64,) option for a 62-class problem, ReLU is generally a strong default for hidden-layer non-linearities, and while Adam is often faster to converge than plain SGD, SGD with a comparatively large learning rate (0.01) and early stopping was evidently able to reach a better validation accuracy within the limited iteration budget used here — a reminder that “Adam usually converges faster” is a general tendency, not a guarantee, and that the specific interaction between learning rate, batch size, and optimizer matters more than any single hyperparameter in isolation. Even the tuned MLP’s accuracy (40.5%) is modest for a 62-class problem, which is unsurprising given the small dataset (≈ 44 images per class on average) relative to the number of classes and the 1,024-dimensional input space — this combination makes overfitting a real risk. The search used `early_stopping=True` with a 10% internal validation

split precisely to guard against this: each candidate model stops training as soon as its internal validation score stops improving for 5 consecutive iterations, rather than always running the full 100-iteration budget. The reported training loss curve (Figure showing training loss vs. epochs) should still be inspected to confirm it is decreasing smoothly rather than oscillating; if overfitting is still suspected (e.g., training loss much lower than validation/test performance would suggest), further mitigations such as L2 regularization (**alpha**), a lower learning rate with more patience, or data augmentation (e.g., small rotations/shifts of the character images) would likely help.

11 Conclusion

This experiment compared a from-scratch, one-vs-all Perceptron (PLA) against a hyperparameter-tuned Multilayer Perceptron (MLP) on a 62-class handwritten character recognition task. The MLP substantially outperformed PLA across all evaluation metrics (accuracy 0.405 vs. 0.223, F1-score 0.411 vs. 0.221), confirming that non-linear decision boundaries and learned feature interactions are necessary for this problem — a linear, step-activation classifier simply cannot separate 62 overlapping character classes well in raw pixel space. The best MLP configuration found by a 10-candidate randomized search — two hidden layers (128, 64), ReLU activation, SGD optimizer, learning rate 0.01, batch size 128 — was identified in just 8.2 seconds thanks to a bounded search budget and early stopping, in contrast to the original exhaustive grid search which was too slow to complete. Even so, the tuned MLP’s accuracy remains modest in absolute terms, reflecting the difficulty of the task given a relatively small dataset (3,410 images across 62 classes) and a high-dimensional flattened-pixel representation; techniques such as convolutional architectures (which exploit spatial structure that flattening discards), data augmentation, or a larger/less time-constrained hyperparameter search would be natural next steps to improve results further.

12 References

1. F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
2. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning Representations by Back-Propagating Errors,” *Nature*, vol. 323, pp. 533–536, 1986.
3. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
4. Kaggle, “English Handwritten Characters Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>